Lesson 5

Material

https://github.com/unixfile/learning-python-2026

Today

- Defining functions and arguments
- Default argument values
- Scope: where Python looks for a name
- `*args` and `**kwargs`
- Lambda expressions
- Comprehensions and generator expressions
- Pythonic style

1 Defining functions

```
# Definition
def multiply(a, b):
    """Return the product of two numbers."""
    return a * b

# Invocation (calling)
multiply(3, 4)    # 12
multiply(25, 25)  # 625
```

- `def` declares a function; `return` sends a value back
- A function with no `return` returns `None`

2 Arguments

```
def greet(name, age, greeting='Hello'):
    print(f'{greeting}, {name}! You are {age} years old.')

greet('Alice', 30)
# Hello, Alice! You are 30 years old.

greet(greeting='Good afternoon', age=80, name='John')
# Good afternoon, John! You are 80 years old.

greet('Taylor', 34, greeting='Hi')
# Hi, Taylor! You are 34 years old.
```

- Positional args go by order; named args make order irrelevant
- `greeting` has a default, used when not supplied
- When mixing, positional arguments must come first

3 Scope: where Python looks for a name

The same name can exist in several places. Python searches them from the inside out and uses the first one it finds:

1. **L**ocal: inside the current function
2. **E**nclosing: a function wrapped around it
3. **G**lobal: the top level of the file
4. **B**uilt-in: names Python provides, like `print`, `len`, `max`

- The first match wins; the search stops there
- This inside-out order is nicknamed the **LEGB rule**

4 Scope in practice

```
x = "global"

def outer():
    x = "enclosing"
    def inner():
        x = "local"
        print(x)    # local
    inner()
    print(x)        # enclosing

outer()
print(x)            # global

print(max)    # <built-in function max>
max = 3       # shadows the built-in!
print(max)    # 3
```

5 The * and ** operators

```
def total(*args):       # collect into a tuple
    return sum(args)

total(1, 2, 3)          # 6

nums = [1, 2, 3]
total(*nums)            # 6  (unpack the list)

def show(**kwargs):     # collect into a dict
    print(kwargs)

show(a=1, b=2)          # {'a': 1, 'b': 2}

d = {'a': 1, 'b': 2}
show(**d)               # same, by unpacking d
```

- In a def, *args/**kwargs collect extra arguments
- In a call, */** unpack a list/dict into arguments

6 Lambda expressions

```
# These two are equivalent
def power(x, y):
    return x ** y

power = lambda x, y: x ** y

people = {'Bob': 25, 'Dave': 20, 'Alice': 30}

# Sort items by value (age)
sorted(people.items(), key=lambda item: item[1])
# [('Dave', 20), ('Bob', 25), ('Alice', 30)]
```

- A lambda is a small, anonymous, one-expression function
- Handy as a throw-away `key=` argument to `sorted`, `max`, etc.

7 List comprehensions

```
In : squares = [x**2 for x in range(6)]
In : squares
Out: [0, 1, 4, 9, 16, 25]

In : evens = [x for x in range(10) if x % 2 == 0]
In : evens
Out: [0, 2, 4, 6, 8]
```

- Concise syntax for building lists from iterables

8 Generator expressions

```
In : squares = (x**2 for x in range(100_000_000))
In : type(squares)
Out: generator
```

- `()` instead of `[]` — values produced on demand
- Takes almost no memory
- Can only be iterated once

```
In : next(squares)
Out: 0
In : next(squares)
Out: 1
```

9 Set comprehensions

```
In : squares = {x**2 for x in range(6)}
In : squares
Out: {0, 1, 4, 9, 16, 25}

In : evens = {x for x in range(10) if x % 2 == 0}
In : evens
Out: {0, 2, 4, 6, 8}
```

- Same syntax as list comprehensions but with {}

10 Dictionary comprehensions

```
In : squares = {x: x**2 for x in range(5)}
In : squares
Out: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

In : original = {'äpple': 1, 'banan': 2, 'körsbär': 3}
In : swapped = {v: k for k, v in original.items()}
In : swapped
Out: {1: 'äpple', 2: 'banan', 3: 'körsbär'}
```

11 Pythonic style

```
# Preferred                          # Avoid
if value is None: ...                 # if value == None: ...
if value is not None: ...             # if value != None: ...
if my_list: ...                       # if len(my_list) > 0: ...
if not my_list: ...                   # if len(my_list) == 0: ...
if condition: ...                     # if condition == True: ...
```

- A linter like `ruff` can flag these automatically

Further reading: the mutable default trap

```
def add_item(item, basket=[]):    # evaluated once!
    basket.append(item)
    return basket

add_item('a')    # ['a']
add_item('b')    # ['a', 'b']   reused!
```

```
def add_item(item, basket=None):
    if basket is None:
        basket = []
    basket.append(item)
    return basket

add_item('a')    # ['a']
add_item('b')    # ['b']
```

- A default value is evaluated **once**, when the function is defined
- Use `None` as the default and build the mutable object inside